

Spring MVC (Web) vs Spring WebFlux (Reactive)

Learning Objectives

بنهاية هذا الدرس سيكون الطالب قادرًا على:

- فهم الفرق بين Spring MVC و Spring WebFlux.
- معرفة متى يستخدم كل منهما.
- فهم سبب اعتماد Spring Cloud Gateway على WebFlux.
- إنشاء مشروعات من Spring Initializr:
 - Spring MVC Application
 - Spring Cloud Gateway (Reactive)
- فهم الأخطاء الشائعة عند خلط Web و WebFlux.

الجزء الأول

ما هو Spring Web؟

عندما نضيف:

spring-boot-starter-web

فنحن نستخدم **Spring MVC**.

يعتمد على:

Servlet API

ويستخدم عادة:

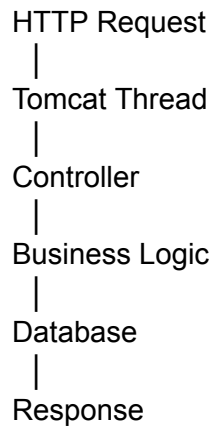
Tomcat

كـ **Embedded Server**.

طريقة تنفيذ الطلب

Client

|



كل Request يحصل على Thread مستقل.

إذا انتظر Database لمدة 5 ثوانٍ:

Thread
↓

Blocked

يبقى الـ Thread محجوزاً طوال فترة الانتظار.

مثال

start.spring.io

Project

Maven

Language

Java

Spring Boot

3.3.x

Dependencies

Spring Web
Spring Boot Actuator

Controller

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public String getProducts() {

        return "Products";
    }
}
```

Application

```
@SpringBootApplication
public class ProductApplication {

    public static void main(String[] args) {

        SpringApplication.run(ProductApplication.class,args);

    }
}
```

تشغيل المشروع

Tomcat started on port 8080

الجزء الثاني

ما هو Spring WebFlux؟

عندما نضيف:

spring-boot-starter-webflux

نستخدم Reactive Programming.

بدلاً من:

Servlet

يستخدم:

Reactive Streams

ويستخدم غالباً:

Netty

بدلاً من Tomcat.

تشغيل المشروع

ستلاحظ:

Netty started on port 8080

وليس

Tomcat started...

كيف يعمل؟

بدلاً من

Thread لكل Request

يستخدم Event Loop.

Event Loop

Request 1

Request 2

Request 3

Request 4

Request 5



Non Blocking

لو Database انتظرت 5 ثوانٍ

الـ Thread لا يبقى محجوزاً.

بل يكمل تنفيذ Requests أخرى.

Reactive Types

بدلاً من

Product

أصبح لدينا

Mono<Product>

و

Flux<Product>

Mono

يعني:

أو 1 قيمة 0

مثال

```
@GetMapping("/{id}")
public Mono<Product> getProduct() {

    return repository.findById(...);

}
```

Flux

يعني

إلى عدد لا نهائي 0

مثال

```
@GetMapping
public Flux<Product> getProducts() {

    return repository.findAll();

}
```

مقارنة

MVC

```
@GetMapping  
public List<Product> getProducts()
```

Reactive

```
@GetMapping  
public Flux<Product> getProducts()
```

MVC

```
@GetMapping("/{id}")  
public Product getProduct()
```

Reactive

```
@GetMapping("/{id}")  
public Mono<Product> getProduct()
```

ماذا يحدث داخل السيرفر؟

Spring MVC

Client

↓

Tomcat Thread

↓

Controller

↓

Repository

↓

Response

Thread محجوز.

Client



Event Loop



Controller



Mono / Flux



Response

لا يتم حجز Thread أثناء الانتظار.

لماذا يستخدم Gateway WebFlux؟

تخيل لدينا

1000 Request

Gateway لا ينفذ Business Logic.

هو فقط:

Receive Request



Authentication



Logging



Routing

↓

Forward Request

كلها عمليات قصيرة.

لذلك WebFlux مثالي.

لماذا لا نستخدم MVC في Gateway؟

Gateway يحتاج:

Thousands of concurrent requests

Reactive مناسب جداً لذلك.

المشروع الأول

Spring MVC

start.spring.io

Dependencies

Spring Web
Actuator

Controller

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public List<String> products(){

        return List.of("Book","Phone");

    }
}
```



```
}
```

تشغيل

Tomcat started...

المشروع الثاني

Reactive

start.spring.io

Dependencies

Spring Reactive Web
Actuator

Controller

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public Flux<String> products(){

        return Flux.just("Book","Phone");

    }

}
```

تشغيل

Netty started...

API Gateway

start.spring.io

Project

Maven

Java

21

Dependencies

Reactive Gateway
Eureka Discovery Client
Config Client
Actuator

ملاحظة حسب الإصدار: في Spring Boot 3.3.x مع Spring Cloud 2023.x يكفي غالبًا استخدام `spring-cloud-starter-gateway`. أما في الإصدارات الأحدث (مثل Spring Boot 4.x مع Spring Cloud 2025.x) ستجد Starter مخصصًا لـ WebFlux، كما لاحظت أثناء تجربتك العملية.

Main Class

```
@SpringBootApplication
public class ApiGatewayApplication {

    public static void main(String[] args) {

        SpringApplication.run(ApiGatewayApplication.class,args);

    }

}
```

Logging Filter

```
@Component
@Slf4j
public class LoggingFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                            GatewayFilterChain chain) {

        log.info("{} {}",
            exchange.getRequest().getMethod(),
            exchange.getRequest().getURI().getPath());

        return chain.filter(exchange);

    }

}
```

```
@Override
public int getOrder() {
    return Ordered.HIGHEST_PRECEDENCE;
}
}
```

Gateway Configuration

```
spring:
cloud:
gateway:
routes:
- id: product-service
uri: lb://PRODUCT-SERVICE
predicates:
- Path=/api/products/**
```

إذا كنت تستخدم الإصدارات الأحدث من Spring Boot 4.x و Spring Cloud 2025.x، فقد تجد أن تعريف الـ routes أصبح تحت:

```
spring:
cloud:
gateway:
server:
webflux:
routes:
```

لذلك يجب دائماً مراجعة توثيق الإصدار الذي تعمل عليه وعدم الاعتماد على أمثلة لإصدارات مختلفة.

لماذا لا نضيف Spring Web؟

إذا أضفت:

```
spring-boot-starter-web
```

مع

```
spring-cloud-starter-gateway
```

فأنت تخطئ بين:

Servlet Stack

و

وقد يؤدي ذلك إلى تعارض عند التشغيل أو إلى استخدام Stack مختلف عما نتوقع.

مقارنة سريعة

الخاصية	(Spring MVC (Web	(Spring WebFlux (Reactive
Programming Model	Blocking	Non-Blocking
Server الافتراضي	Tomcat	Netty
Return Type	Object / List	Mono / Flux
Thread Model	Thread لكل Request	Event Loop
مناسب للتطبيقات التقليدية	✓	✓ (إذا كان باقي النظام Reactive)
مناسب لـ API Gateway	✗ ليس الخيار المعتاد	✓ الخيار الموصى به
Spring Cloud Gateway	غير مبني عليه	مبني عليه

Spring Cloud Gateway Filters

Understanding StripPrefix & Weight

Learning Objectives

بنهاية هذا الدرس سيكون الطالب قادرًا على:

- فهم وظيفة `StripPrefix`.
- معرفة متى يستخدم `StripPrefix`.
- فهم وظيفة `Weight`.
- معرفة استخدام `Weight` في Canary Deployment.
- التمييز بين Routing العادي و Weighted Routing.

الجزء الأول

StripPrefix

ما هو StripPrefix؟

`StripPrefix` هو **Gateway Filter** يقوم بحذف عدد معين من أجزاء الـ URL قبل إرسال الطلب إلى الـ Backend Service.

بمعنى آخر:

Gateway يستقبل URL من العميل، ثم يزيل جزءًا من المسار قبل إرسال الطلب إلى الخدمة.

لماذا نحتاجه؟

نفترض أن جميع الخدمات تعمل خلف Gateway.

العميل يرسل دائماً:

/api/products/10

لكن Product Service لا يعرف شيئاً عن:

/api

بل يتوقع فقط:

/products/10

أو ربما:

/10

هنا يأتي دور StripPrefix.

مثال 1

Gateway Configuration

```
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE
      predicates:
        - Path=/api/products/**
```

العميل يرسل:

GET /api/products/10

Gateway يرسل إلى Product Service:

GET /api/products/10

لأنه لم يتم حذف أي جزء.

مثال 2

إضافة

```
filters:
  - StripPrefix=1
```

الطلب:

/api/products/10

يصبح:

/products/10

لأن الجزء الأول:

api

تم حذفه.

مثال 3

filters:

- StripPrefix=2

الطلب:

/api/products/10

يصبح:

/10

لأن:

api

products

تم حذفهما.

ماذا يحدث؟

بدون StripPrefix

Client

↓

GET /api/products/10

↓

Gateway



GET /api/products/10



Product Service

مع StripPrefix=1

Client



GET /api/products/10



Gateway



GET /products/10



Product Service

مع StripPrefix=2

Client



GET /api/products/10



Gateway



GET /10



Product Service

متى نستخدمه؟

مثال:

Gateway

/api/products/**

لكن داخل الخدمة

```
@RestController  
@RequestMapping("/products")
```

هنا نستخدم

StripPrefix=1

مثال آخر

Gateway

/api/products/**

لكن Controller

```
@RestController  
@RequestMapping("/")
```

أو

```
@GetMapping("/{id}")
```

فنستخدم

StripPrefix=2

متى لا نستخدمه؟

إذا كان Product Service نفسه يحتوي:

@RequestMapping("/api/products")

فإن:

StripPrefix

غير مطلوب.

بل قد يسبب 404.

ملخص StripPrefix

الطلب بعد التعديل	الطلب الأصلي	قيمة StripPrefix
api/products/10/	api/products/10/	0
products/10/	api/products/10/	1
10/	api/products/10/	2

الجزء الثاني

Weight Predicate

ما هو Weight؟

Weight هو **Route Predicate** يسمح بتوزيع الطلبات بين أكثر من Route بنسبة معينة.

ليس كل الطلبات تذهب إلى نفس الخدمة.

بل يتم تقسيمها.

مثلاً

لدينا إصداران من Product Service

Product Service v1

Product Service v2

ولا نريد إرسال جميع المستخدمين مباشرة إلى الإصدار الجديد.

لماذا؟

لأن الإصدار الجديد قد يحتوي على أخطاء.

بدلاً من:

100%

↓

Version 2

نفعل:

80%

↓

Version 1

20%

↓

Version 2

يسمى هذا

Canary Deployment

مثال

routes:

- id: product-v1
uri: lb://PRODUCT-V1
predicates:

- Path=/api/products/**
- Weight=products,80

- id: product-v2
uri: lb://PRODUCT-V2
predicates:

- Path=/api/products/**
- Weight=products,20

لاحظ

products

هو اسم المجموعة.

ويجب أن يكون نفسه في جميع الـ Routes.

ماذا تعني 80 و 20؟

ليس أول 80 طلبًا.

وليس أول 80 مستخدمًا.

بل متوسط التوزيع.

مثلاً لو وصل:

1000 Request

تقريبًا

800

↓

Product V1

و

200

↓

Product V2

ماذا يحدث؟



مثال عملي

نفترض أننا طورنا نسخة جديدة.
بدلاً من

100%



Version 2

نبدأ بـ

95%



Version 1

5%



Version 2

إذا لم تظهر مشاكل
نجعلها

80%



Version 1

20%



Version 2

تم

50%



Version 1

50%



Version 2

تم

0%



Version 1

100%



Version 2

بدون توقف الخدمة.

الفرق بين Weight و Load Balancing

قد يظن الطالب أن Weight مثل Load Balancer.

لكن هناك فرق كبير.

Load Balancer

لدينا نسختان متطابقتان.

Product Instance 1

Product Instance 2

والهدف

زيادة الأداء

كل Instance يحتوي نفس الكود.

Weight

لدينا

Version 1

Version 2

نسختان مختلفتان.

والهدف

تجربة الإصدار الجديد تدريجيًا.

مقارنة

Load Balancer

Client

↓

Gateway

↓

Product v1

↓

Instance 1

Instance 2

Instance 3

كلها نفس الإصدار.

Weight

Client

↓

Gateway

↓

80%

↓

Product V1

20%

↓

Product V2

الإصداران مختلفان.

مثال كامل

spring:

cloud:

gateway:

routes:

- id: product-v1
uri: lb://PRODUCT-V1
predicates:
 - Path=/api/products/**
 - Weight=products,80
- id: product-v2
uri: lb://PRODUCT-V2
predicates:
 - Path=/api/products/**

- Weight=products,20

متى نستخدم Weight؟

- Canary Deployment
- Blue/Green Deployment (في بعض السيناريوهات)
- اختبار إصدار جديد مع نسبة صغيرة من المستخدمين
- A/B Testing للخدمات

متى لا نستخدمه؟

إذا كان لدينا نسخة واحدة فقط من الخدمة.

أو إذا كانت النسخ متطابقة تمامًا؛ ففي هذه الحالة يكون **Load Balancer** هو الأداة المناسبة.

مقارنة نهائية

Weight	StripPrefix	الخاصية
Route Predicate	Gateway Filter	النوع
توزيع الطلبات بين عدة Routes	تعديل الـ URL قبل إرساله	الوظيفة
اختيار الـ Route	Path	يؤثر على
A/B Testing و Canary Deployment	إزالة Prefix مثل <code>api/</code>	الاستخدام الأشهر
80% إلى V1 و 20% إلى V2	<code>api/products/10 → /products/10</code>	مثال